

ASSIGNMENT – 5

Simulation of ARP (Address Resolution Protocol) Using Socket Programming

AIM

To simulate the Address Resolution Protocol (ARP) using socket programming. The server acts as a host or router: it enters the packet details (source IP, source MAC, destination IP and 16-bit data), develops an ARP Request packet containing SourceIP | SourceMAC | DestinationIP, broadcasts it to all connected clients, and waits for an ARP Reply. When a reply carrying the Destination MAC address arrives, the server sends the packet to the matching destination with the format SourceIP | SourceMAC | DestinationIP | DestinationMAC | 16-bit data.

The client enters its own IP and MAC address. When an ARP Request is received it checks whether the Destination IP is its own IP; if not it stays silent, if yes it replies with SourceIP | SourceMAC | DestinationIP | DestinationMAC and then receives and displays the packet from the server. The validity of IP and MAC addresses is checked before any packet is processed.

ALGORITHM

- SERVER: validate destination IP, source IP, source MAC and 16-bit data.
- SERVER: accept three clients, develop the ARP Request packet and broadcast it.
- SERVER: poll for an ARP Reply, print the destination MAC and send the packet to it.
- CLIENT: enter own IP and MAC; read the ARP Request and compare Destination IP with own IP.
- CLIENT: if the IP matches, send the ARP Reply and display the packet from the server.
- CLIENT: if the IP does not match, stay silent and send no reply.

SERVER CODE (HOST / ROUTER) - arp_server.py

```
import re
import socket
import sys
import time

# arp server (host / router). broadcasts an ARP request to all connected clients,
# waits for the ARP reply, then sends the packet to the matching destination.
# usage: arp_server.py <dest_ip> <src_ip> <src_mac> <data_16bit> <num_clients>

def is_valid_ip(ip):
    # check if the IP is four numbers 0-255 separated by dots
    parts = ip.split(".")
    if len(parts) != 4:
        return False
    for p in parts:
        if not p.isdigit():
```

```
        return False
    if int(p) < 0 or int(p) > 255:
        return False
    return True

def is_valid_mac(mac):
    # check MAC like XX-XX-XX-XX-XX-XX with hex digits
    pattern = r"^[0-9A-Fa-f]{2}(-[0-9A-Fa-f]{2}){5}$"
    return re.match(pattern, mac) is not None

dest_ip, src_ip, src_mac, data, num = sys.argv[1], sys.argv[2], sys.argv[3], sys.argv[4], int(sys.argv[5])

print("Enter the details of packet received.")
print(f"Destination IP : {dest_ip}")
print(f"Source IP : {src_ip}")
print(f"Source MAC : {src_mac}")
print(f"16 bit data : {data}")

# check the validity of IP and MAC addresses
if not (is_valid_ip(dest_ip) and is_valid_ip(src_ip)):
    print("Invalid IP address. Packet rejected.")
    sys.exit()
if not is_valid_mac(src_mac) or len(data) != 16 or any(c not in "01" for c in data):
    print("Invalid MAC address or data. Packet rejected.")
    sys.exit()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("127.0.0.1", 5007))
server.listen(5)

clients = []
print("Waiting for clients to connect...")
for _ in range(num):
    conn, addr = server.accept()
    conn.settimeout(0.5)
    clients.append(conn)
print(f"{num} clients connected.")

req = f"{src_ip} | {src_mac} | {dest_ip}"
print("Developing ARP Request packet")
print(req)
for c in clients:
    # broadcast the ARP request to every client
    c.sendall(f"ARP_REQUEST|{req}\n".encode())
print("The ARP Request packet is broadcasted.")
print("Waiting for ARP Reply...")

reply = None
deadline = time.time() + 8
while time.time() < deadline and reply is None:
    for c in clients:
```

```
        try:
            line = c.recv(1024).decode().strip()
        except socket.timeout:
            continue
        if not line:
            continue
        if line.startswith("ARP_REPLY"):
            reply = (c, line)
            break

    if reply:
        conn, msg = reply
        print("ARP Reply received")
        print(msg.split("|", 1)[1].strip())
        dest_mac = msg.split("|")[4].strip()
        print(f"Sending the packet to : {dest_mac}")
        packet = f"{src_ip} | {src_mac} | {dest_ip} | {dest_mac} | {data}"
        print("Packet Sent:", packet)
        conn.sendall(f"PACKET|{packet}\n".encode())
        time.sleep(0.5)
    else:
        print("No ARP Reply received within the timeout.")

for c in clients:
    c.close()
server.close()
```

CLIENT CODE (END HOST) - arp_client.py

```
import socket
import sys

# arp client (end host). checks incoming ARP requests and replies only when the
# destination IP matches its own IP, then receives the packet from the server.
# usage: arp_client.py <own_ip> <own_mac> <client_no>

own_ip, own_mac, cid = sys.argv[1], sys.argv[2], sys.argv[3]
print(f"Enter the IP address : {own_ip}")
print(f"Enter the Mac address : {own_mac}")

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5007))
f = client.makefile("r", encoding="utf-8")

line = f.readline().strip()
parts = line.split("|")
if parts[0] == "ARP_REQUEST":
```

```
req = " | ".join(p.strip() for p in parts[1:4])
print(f"ARP Request Received : {req}")
src_ip, src_mac, dest_ip = [p.strip() for p in parts[1:4]]
if dest_ip == own_ip:
    # our own IP is being resolved, send the ARP reply
    print("IP address matches")
    reply = f"ARP_REPLY|{src_ip} | {src_mac} | {dest_ip} | {own_mac}"
    print("ARP Reply Sent :", " | ".join(p.strip() for p in reply.split("|")[1:]))
    client.sendall((reply + "\n").encode())
    # now wait for the actual packet from the server
    line2 = f.readline().strip()
    parts2 = line2.split("|")
    if parts2[0] == "PACKET":
        print("Received Packet is :", " | ".join(p.strip() for p in parts2[1:]))
    else:
        print("IP address does not match.")

client.close()
```

SERVER OUTPUT

```
Enter the details of packet received.
Destination IP : 155.157.65.128
Source IP : 123.128.34.56
Source MAC : AF-45-E5-00-97-12
16 bit data : 1011110000101010
Waiting for clients to connect...
3 clients connected.
Developing ARP Request packet
123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128
The ARP Request packet is broadcasted.
Waiting for ARP Reply...
ARP Reply received
123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128 | 45-DA-62-21-1A-B2
Sending the packet to : 45-DA-62-21-1A-B2
Packet Sent: 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128 | 45-DA-62-21-1A-B2 | 1011110000101010
```

CLIENT 1 OUTPUT (IP DOES NOT MATCH)

```
Enter the IP address : 165.43.158.158
Enter the Mac address : 09-DF-90-26-6C-09
ARP Request Received : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128
IP address does not match.
```

CLIENT 2 OUTPUT (IP MATCHES, PACKET RECEIVED)

```
Enter the IP address : 155.157.65.128
Enter the Mac address : 45-DA-62-21-1A-B2
ARP Request Received : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128
IP address matches
ARP Reply Sent : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128 | 45-DA-62-21-1A-B2
Received Packet is : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128 | 45-DA-62-21-1A-B2 | 1011110000101010
```

CLIENT 3 OUTPUT (IP DOES NOT MATCH)

```
Enter the IP address : 15.143.158.18
Enter the Mac address : 19-0F-01-63-C7-D4
ARP Request Received : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128
IP address does not match.
```

VALIDITY CHECK OUTPUT (INVALID IP REJECTED)

```
Enter the details of packet received.
Destination IP : 999.999.999.999
Source IP : 123.128.34.56
Source MAC : AF-45-E5-00-97-12
16 bit data : 1011110000101010
Invalid IP address. Packet rejected.
```

TASKS PERFORMED

Task	Details	Result
Validate addresses	Server checks IP format (0-255 per octet) and MAC format (XX-XX-XX-XX-XX-XX)	Invalid IP 999.999.999.999 rejected before any broadcast
Connect clients	Router server accepts three clients on 127.0.0.1:5007	3 clients connected
ARP Request	Packet 123.128.34.56 AF-45-E5-00-97-12 155.157.65.128 developed and broadcast to all clients	All three clients received the request
No-match clients	Clients 1 and 3 compared the destination IP with their own IP	Both printed "IP address does not match." and stayed silent
Matching client	Client 2 (155.157.65.128) matched the destination IP	ARP Reply with MAC 45-DA-62-21-1A-B2 sent to the server
Packet delivery	Server sent the packet to the replying client with the destination MAC appended	Client 2 displayed the complete 5-field packet

LEARNING OUTCOMES

- I broadcast an ARP request from a router server to all connected clients.
- I implemented the client-side IP match check with silent behaviour for other hosts.
- I delivered the packet only after the destination MAC was resolved.
- I validated IP and MAC formats before processing any packet.
- I observed that all clients must be connected before the broadcast.